

Sakshi Khochare

602-332-4847 | sakshi.khochare@gmail.com | <https://www.linkedin.com/in/sakshi-khochare/> | Tempe, Arizona

EDUCATION

Arizona State University - GPA: 3.77/4.00

Masters of Science in Computer Science

Aug 2023 – May 2025

Tempe, Arizona

Fr. Conceicao Rodrigues College of Engineering - GPA: 8.85/10.00

Bachelors of Engineering in Information Technology

Aug 2017 – May 2021

Mumbai, India

EXPERIENCE

Software Engineering Intern

HCL America Inc, HCLSoftware

January 2025 – Current

Remote - USA

- Built a full-stack SaaS platform using **Micro Frontend Architecture (JavaScript, React, Node.js)** integrated with **BigFix**, enabling remote patch/configuration deployment for 10,000+ managed devices and cutting manual ops effort by approximately 40%.
- Designed backend services in **JavaScript** and **Go** with **PostgreSQL** and **SQLC** for querying, implementing **mTLS** enabled **REST APIs** with **Swagger** and asynchronous telemetry ingestion workers that enabled real-time anomaly detection and system monitoring reducing escalation handling time.
- Built **SSE**-backed real-time update pipelines to keep client-side state consistent across micro-frontends, eliminating stale data issues and reducing state-sync conflicts during interactive workflows.
- Built accessible, production-ready UI components with **React**, **MUI**, custom optimization hooks, and shared **Context/Zustand** state, implementing **WCAG/ARIA**-compliant interaction patterns and keyboard navigation.
- Integrated the system into a **Webpack Module Federation** micro-frontend environment with lazy loading and maintained 90%+ **Lighthouse** accessibility scores.
- Delivered recurring product demos to design/product teams, driving 3+ major UX revisions and helping reduce user-reported friction in early pilot tests.

Application Development Associate

Accenture Pvt Ltd

June 2021 – July 2023

Mumbai, India

- Built a **JavaScript/React** patient portal integrated with **Microsoft Dynamics 365** via secure backend APIs, enabling real-time access for 5,000+ active patient accounts and improving case-processing speed by approximately 25%.
- Designed the portal architecture using **React** as the presentation layer and a **.NET/Node.js** API layer to interface with Dataverse, ensuring strict data security and **HIPAA**-aligned access control.
- Implemented **OAuth2** authentication following **OWASP** best practices to enforce robust authorization, session security, and controlled access to **PHI**.
- Wrote **SQL** queries and derived metrics from high-volume patient datasets to support workflow optimization and data validation across **ETL** pipelines.
- Optimized portal data flows by coordinating with **Informatica** ETL processes for attachments, insurance documents, and high-volume patient datasets.
- Worked across multiple client implementations under tight delivery schedules, driving end-to-end QA cycles and test-script management using **Zephyr/Jira**, ensuring on-time releases with consistently high coverage and clean handoffs.

TECHNICAL SKILLS

Languages: JavaScript, TypeScript, Python, Go, C++, Java, SQL

Frameworks: React.js, Redux, Next.js, Node.js, Express.js, Flask, FastAPI

Cloud: GCP (GKE, Cloud Run, Vertex AI Search), AWS (EC2, S3, Lambda), Azure (Blob, Service Bus)

DevOps & CI/CD: Docker, Kubernetes, Terraform, GitHub Actions, Jenkins, ArgoCD

AI & Agents: LangChain, LangGraph, RAG workflows, OpenTelemetry

Databases: PostgreSQL, MySQL, Oracle, NoSQL familiarity

PROJECTS

- MLOps Pipeline Orchestration (Docker, GCS, ArgoCD, GKE)** — Automated an ML pipeline with Docker and GitOps on GKE, reducing iteration time by **30%**, and added IAM, RBAC, and NetworkPolicies to secure workload access and pod communication. [link](#)
- Scalable Image Recognition App (AWS, Flask, SQS, Python)** — Built a 3-tier image recognition system with Flask, async SQS queues, and EC2 autoscaling, sustaining 1,000 concurrent requests with **99.9%** uptime, and secured the architecture with VPC subnet isolation, routing rules, and least-privilege security groups. [link](#)
- Serverless Face Recognition Application (AWS, Lambda, Python)** — Built an elastic video-to-frame pipeline using Lambda, Docker, and FFmpeg, processing 1,000 images in under **5 seconds**, and secured execution with fine-grained IAM roles and least-privilege access to S3 and ECR. [link](#)